Institución Universitaria

INTELIGENCIA DE NEGOCIOS

Taller evaluativo 2

Proceso ETL con los datasets de Airbnb Bogotá, Distrito Capital, Colombia (20%)

Fecha de entrega: 11 de octubre, 11:59 p. m.
Medio de entrega: Tarea en Teams
Modalidad: Trabajo en grupo

Objetivo

Aplicar los conceptos de **Extracción, Transformación y Carga (ETL)** sobre los datasets de **Bogotá, Distrito Capital, Colombia**, almacenados en una base de datos MongoDB local, mediante la implementación de un proceso automatizado en Python que incluya **manejo de logs**, **análisis exploratorio de datos** y **documentación del flujo de trabajo**.

Ubicación de los datasets fuente

Nota importante: Aunque el desarrollo del taller se realizará sobre la **base de datos MongoDB local**, los datasets requeridos para cargar previamente dicha base de datos se encuentran en la siguiente ubicación del proyecto o repositorio de trabajo.

Country/City	File Name	Description
Bogotá	listings.csv.gz	Detailed Listings data
Bogotá	calendar.csv.gz	Detailed Calendar Data
Bogotá	reviews.csv.gz	Detailed Review Data

Aclaración sobre la carga en MongoDB:
Los archivos fuente proporcionados (listings.csv.gz, calendar.csv.gz, reviews.csv.gz) deben ser utilizados por cada grupo para cargar la base de datos MongoDB local.

Es responsabilidad del grupo:

- crear la base de datos en MongoDB,
- importar los datasets como colecciones (Listings, Reviews y Calendar),
- y validar que la información esté correctamente almacenada antes de iniciar el proceso ETL.

El desarrollo del taller debe realizarse obligatoriamente sobre MongoDB, no directamente sobre los archivos CSV.

No se evaluará como válido un desarrollo que no utilice MongoDB como fuente de datos.

1. Conexión y extracción de datos

Actividades

1.1. Conexión a la base de datos

Conectarse a la base de datos local de MongoDB que contiene, como mínimo, las siguientes colecciones:

- Listings
- Reviews
- Calendar

1.2. Clase de extracción

Crear una clase llamada **Extraccion** en Python que permita:

- establecer conexión con la base de datos,
- consultar cada colección,
- cargar los datos en **DataFrames de pandas**,
- registrar en un log la conexión realizada y la cantidad de registros extraídos por colección.

Entregable

Archivo Python llamado:

- extraccion.py

Este archivo debe contener la clase **Extraccion**, debidamente documentada y funcional.

2. Análisis exploratorio de datos (EDA)

Objetivo

Comprender la estructura, calidad y distribución de los datos antes de realizar las transformaciones.

Actividades en Jupyter Notebook

2.1. Entendimiento general de los datos

Para cada colección:

- mostrar las primeras filas (head()),
- identificar la cantidad de registros y columnas,
- revisar tipos de datos (info()),
- presentar una descripción general de las variables más relevantes.

2.2. Calidad de los datos

Analizar y documentar:

- valores nulos o faltantes por columna,
- registros duplicados,
- necesidad o no de eliminar duplicados,
- posibles valores atípicos en variables como:
    - price
    - minimum_nights
    - availability_365

2.3. Posibles transformaciones

Evaluar y justificar si es necesario:

- desanidar campos complejos o anidados, por ejemplo:
    - amenities
    - información del host
- agrupar o resumir datos, por ejemplo:
    - calendario por mes o semana
- estandarizar formatos de:
    - fecha
    - moneda
    - texto

2.4. Documentación de hallazgos

El Notebook debe incluir explicación de los principales hallazgos, por ejemplo:

- inconsistencias detectadas,
- variables problemáticas,
- correlaciones relevantes,
- outliers,
- decisiones que impactarán la fase de transformación.

Entregable

Archivo Jupyter Notebook llamado:

- exploracion_airbnb.ipynb

Debe incluir:

- código,
- visualizaciones,
- análisis interpretativo.

3. Transformación de datos

Objetivo
Preparar los datos para su carga en una base de datos analítica y para su posterior análisis.

Actividades

3.1. Clase de transformación

Crear una clase llamada **Transformacion** en Python que implemente, como mínimo, las siguientes tareas:

- limpieza de valores nulos y duplicados,
- normalización de precios:
  - eliminar símbolos como $ y ,
  - convertir el campo a valor numérico
- conversión de fechas a formato estándar YYYY-MM-DD,
- derivación de variables a partir del campo date, por ejemplo:
  - año
  - mes
  - día
  - trimestre
- categorización de precios por rangos,
- expansión o tratamiento de campos anidados cuando aplique,
- generación de uno o más DataFrames limpios y listos para la carga.

3.2. Documentación del proceso

Cada transformación debe quedar documentada mediante:

- comentarios,
- docstrings,
- o una explicación clara dentro del código.

3.3. Registro en logs

Integrar logs para registrar, como mínimo:

- transformaciones realizadas,
- cantidad de registros antes y después de la limpieza,
- advertencias o errores encontrados durante el proceso.

Entregable
Archivo Python llamado:

- transformacion.py

Debe contener la clase **Transformacion** funcional y documentada.

4. Carga de datos

Actividades

4.1. Clase de carga

Crear una clase llamada **Carga** que permita:

- insertar los datos transformados en una nueva base de datos **SQLite**,
- exportar los datos transformados a uno o varios archivos **XLSX**,
- verificar que los registros se hayan cargado correctamente,
- registrar en logs los eventos principales del proceso.

Entregable
Archivo Python llamado:

- carga.py

Debe contener la clase **Carga** funcional.

Nota:
Como valor agregado, quienes deseen llevar este proceso a otro sistema gestor de base de datos como **PostgreSQL, MySQL, SQL Server u Oracle**, podrán usarlo posteriormente como base para su proyecto final. Para este trabajo, **SQLite es suficiente y obligatorio**.

5. Manejo de logs

Requerimiento obligatorio
Todos los scripts principales del proceso ETL deben incluir manejo de logs.
Esto aplica para:

- extraccion.py
- transformacion.py
- carga.py

El sistema de logs debe:

- generar un archivo por ejecución, por ejemplo:
  - logs/log_YYYYMMDD_HHMM.txt
- registrar mensajes con niveles como:
  - INFO
  - WARNING
  - ERROR
- incluir fecha, hora y descripción clara del evento.

Importante:
Pueden implementar los logs mediante una clase reutilizable o mediante un módulo centralizado, siempre que el manejo sea claro, consistente y funcional.

6. Informe final

El grupo debe entregar un informe en PDF que incluya como mínimo:

1. Portada
2. Introducción
3. Descripción del dataset
4. Resumen del análisis exploratorio
5. Gráficas y hallazgos principales
6. Descripción de las transformaciones realizadas
7. Ejemplo del log generado
8. Conclusiones sobre la calidad y utilidad de los datos
9. Referencias

Formato de entrega

- **PDF**

7. Entrega en repositorio

Cada grupo deberá subir su proyecto completo a un repositorio público de **GitHub** o **GitLab** con una estructura mínima similar a la siguiente:

```
etl_airbnb/
│
├── src/
├── notebooks/
├── logs/
├── README.md
└── requirements.txt
```

El archivo README.md debe incluir:

- descripción del proyecto y su objetivo,
- instrucciones de instalación:
  - creación del entorno virtual,
  - instalación de dependencias,
  - ejecución del proyecto,
- integrantes del grupo y responsabilidades,
- ejemplo de ejecución del proceso ETL.
-

8. Entrega final

Cada grupo deberá entregar en **Teams**:

- el **informe en PDF**,
- el **enlace al repositorio público**.

Importante
No basta con subir únicamente el PDF o únicamente el repositorio. La entrega debe estar completa y organizada.

9. Criterios mínimos esperados

Para que el trabajo se considere bien desarrollado, como mínimo se espera que el trabajo incluya:

- conexión correcta a MongoDB,
- extracción funcional de las colecciones,
- análisis exploratorio con interpretación,
- transformaciones justificadas y visibles en el código,
- carga correcta a SQLite,
- exportación a XLSX,
- logs funcionales,
- repositorio organizado,
- informe coherente con lo implementado.

10. Rúbrica de evaluación

Criterio	Excelente (5)	Satisfactorio (4)	Básico (3)	Insuficiente (1-2)	Pond.
Exploración y visualización de datos	EDA completo, con gráficas interpretadas y hallazgos claros	EDA con gráficas básicas y descripciones parciales	EDA superficial, con poca interpretación	No realiza análisis ni visualizaciones	25%
Transformaciones aplicadas	Limpieza completa, transformaciones correctas, justificadas y consistentes con el análisis	Transformaciones adecuadas con leves fallos	Limpieza parcial o mal documentada	No transforma o genera errores	25%
Implementación en Python (ETL + logs)	Código funcional, modular, documentado y bien organizado	Código funcional pero con limitada modularidad o documentación	Código con errores, baja organización o poca documentación	Código incompleto o inejecutable	25%
Informe final	Redacción clara, análisis crítico y evidencias completas	Informe correcto pero poco analítico	Informe superficial o incompleto	No entrega informe o no corresponde al trabajo	15%
Organización y presentación	Repositorio ordenado, buena presentación y evidencias coherentes del trabajo en equipo	Presentación comprensible y organizada	Organización deficiente o estructura incompleta	Trabajo confuso o incompleto	10%

11. Recomendaciones importantes

- No dejen la transformación únicamente en el Notebook. El proceso ETL formal debe quedar implementado en los archivos Python solicitados.
- No entreguen un repositorio desordenado. La estructura debe ser clara y fácil de revisar.

- No documenten solo por cumplir. Se espera que expliquen por qué transformaron los datos de cierta manera.
- Las gráficas del EDA deben tener sentido analítico. No se trata solo de poner imágenes, sino de interpretar resultados.
- Antes de entregar, validen que el proyecto realmente funcione en otro equipo siguiendo el README.md.